

MetaDAO

Security Assessment

February 17th, 2026 — Prepared by OtterSec

Akash Gurugunti sud0u53r.ak@osec.io

Mingxuan Yang ymxf00l@osec.io

Filippo Barsanti barsa@osec.io

Table of Contents

Executive Summary **2**

Overview 2

Key Findings 2

Scope **3**

Findings **4**

Vulnerabilities **5**

OS-MCV-ADV-00 | Improper Truncation of Liquidity Amounts 7

OS-MCV-ADV-01 | AMM Invariant Violation Due to Zero Liquidity 8

OS-MCV-ADV-02 | PDA Authority Mismatch 9

OS-MCV-ADV-03 | Pool Initialization with Zero Base Reserves 10

OS-MCV-ADV-04 | Lack of Dust Collection Mechanism 11

OS-MCV-ADV-05 | Vault Lock Due to Excessive Outcome Count 12

General Findings **13**

OS-MCV-SUG-00 | Inconsistent DAO Account Boxing 14

OS-MCV-SUG-01 | Code Optimization 15

OS-MCV-SUG-02 | Missing Validation Logic 16

OS-MCV-SUG-03 | Code Refactoring 18

OS-MCV-SUG-04 | Code Maturity 19

Appendices

Vulnerability Rating Scale **21**

Procedure **22**

© 2026 Otter Audits LLC. All Rights Reserved.

01 — Executive Summary

Overview

Metadao engaged OtterSec to assess the `conditional-vault` and `futarchy` programs. This assessment was conducted between October 29th, 2025 and February 12th, 2026. For more information on our auditing methodology, refer to Appendix B.

Key Findings

We produced 11 findings throughout this audit engagement.

In particular, we identified a high-risk vulnerability where the AMM position PDA is derived from position authority but is initialized with liquidity provider as the stored authority, allowing anyone to create a denial-of-service scenario by pre-initializing an AMM position (OS-MCV-ADV-02). Also, the AMM allows initial liquidity to be created with zero base tokens, breaking pool invariants (OS-MCV-ADV-03), and integer division during redemptions may result in small dust amounts in the vault, which may accumulate over time and lock a non-trivial amount of value due to the absence of any dust collection logic (OS-MCV-ADV-04).

Additionally, if minimum futarchic liquidity thresholds are set to zero, an attacker may manipulate spot reserves to one, resulting in integer truncation to initialize pass/fail pools with zero reserves, breaking AMM invariants and freezing the DAO (OS-MCV-ADV-01). Furthermore, a vault may be initialized with a number of outcomes that later results in split, merge, and redeem operations to exceed Solana's account limit, rendering the vault unusable (OS-MCV-ADV-05).

We also made recommendations to optimize the code by eliminating redundant ATA validations in high-frequency instructions, relaxing unnecessary ATA constraints, closing unutilized conditional vault accounts after proposal finalization, and short-circuiting arbitrage execution when no profitable opportunity exists (OS-MCV-SUG-01). We further suggested, adding overflow checks, enforcing a non-zero value to preserve oracle liveness, and preventing split/merge execution after proposal resolution to maintain proper state transitions and governance correctness (OS-MCV-SUG-02). Moreover, we advised incorporating certain refactors to mitigate potential security issues and improve functionality (OS-MCV-SUG-03).

© 2026 Otter Audits LLC. All Rights Reserved.

02 — Scope

The source code was delivered to us in a Git repository at https://github.com/metaDAOproject. This audit was performed against 841f887.

Brief descriptions of the programs are as follows:

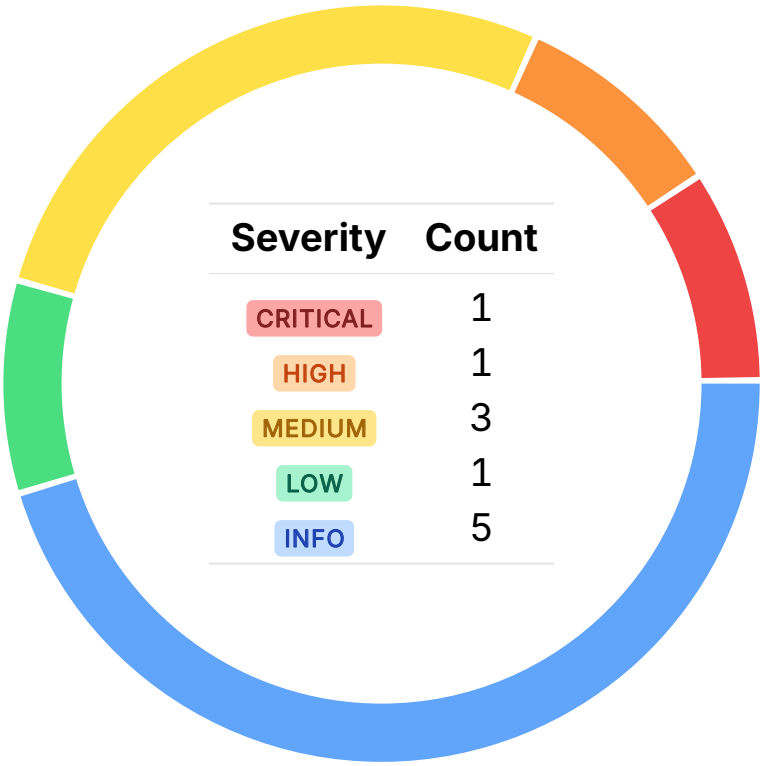
Name	Description
conditional-vault	It implements a binary conditional token system for MetaDAO, enabling users to lock collateral, mint paired outcome tokens, trade them freely, and redeem the winning outcome after a question is resolved.
futarchy	It replaces DAO voting with prediction markets by creating PASS/FAIL markets for each proposal and using market prices to decide outcomes. Traders stake real capital in conditional AMMs, and a proposal passes only if the PASS market's TWAP exceeds the FAIL market by a defined threshold.

© 2026 Otter Audits LLC. All Rights Reserved.

03 — Findings

Overall, we reported 11 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



Severity	Count
CRITICAL	1
HIGH	1
MEDIUM	3
LOW	1
INFO	5

© 2026 Otter Audits LLC. All Rights Reserved.

04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities we identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in Appendix A.

ID	Severity	Status	Description
OS-MCV-ADV-00	CRITICAL	RESOLVED ✓	Liquidity withdrawal math truncates `u128` values to `u64`, corrupting proportional calculations once the total liquidity exceeds `u64::MAX`.
OS-MCV-ADV-01	HIGH	RESOLVED ✓	If minimum futarchic liquidity thresholds are set to zero, an attacker may manipulate spot reserves to 1, resulting in integer truncation to initialize pass/fail pools with zero reserves, breaking AMM invariants and freezing the DAO in futarchy state.
OS-MCV-ADV-02	MEDIUM	RESOLVED ✓	The `amm_position` PDA is derived from `params.position_authority` but was initialized with `liquidity_provider` as the stored authority, allowing anyone to create a temporary denial-of-service scenario by providing liquidity to an `AmmPosition`.
OS-MCV-ADV-03	MEDIUM	RESOLVED ✓	The AMM allows initial liquidity to be created with zero base tokens, breaking pool invariants.
OS-MCV-ADV-04	MEDIUM	RESOLVED ✓	Integer division during redemptions may result in small dust amounts in the vault, which may accumulate over time and lock a non-trivial amount of value due to the absence of any dust collection logic.

© 2026 Otter Audits LLC. All Rights Reserved.

OS-MCV-ADV-05	LOW	RESOLVED	A vault may be initialized with a number of outcomes that later results in split, merge, and redeem operations to exceed Solana's account limit, rendering the vault unusable.

© 2026 Otter Audits LLC. All Rights Reserved.

Improper Truncation of Liquidity Amounts CRITICAL OS-MCV-ADV-00

Description

Within the `withdraw_liquidity` instruction, while calculating the withdrawable base and quote amount, `dao.amm.total_liquidity` and `liquidity_to_withdraw` are tracked as `u128`, but are cast to `u64` before they are passed into `get_base_and_quote_withdrawable`. Thus, when `total_liquidity` exceeds `u64::MAX`, the cast silently wraps modulo 2^{64}, resulting in the proportional share calculation to utilize a much smaller denominator. This will result in inflated base/quote withdrawal amounts as long as the base/quote withdraw amounts within the limits of base/quote reserves.

>_ *futarchy/src/instructions/withdraw_liquidity.rs* RUST

```rust
impl WithdrawLiquidity<'_> {
    pub fn handle(ctx: Context<Self>, params: WithdrawLiquidityParams) -> Result<()> {
        [...]
        let (base_to_withdraw, quote_to_withdraw) = {
            let PoolState::Spot { ref spot } = dao.amm.state else {
                // TODO: check that pool is already in right state
                unreachable!();
            };
            spot.get_base_and_quote_withdrawable(
                liquidity_to_withdraw as u64,
                total_liquidity as u64,
            )
        };
        [...]
    }
}
```

Remediation

Update the logic to prevent the truncation of liquidity amounts to `u64`.

Patch

Identified and resolved by client in PR#415.

© 2026 Otter Audits LLC. All Rights Reserved.

AMM Invariant Violation Due to Zero Liquidity HIGH OS-MCV-ADV-01

Description

The current implementation does not prevent setting `min_base_futarchic_liquidity` and `min_quote_futarchic_liquidity` to zero. There exist multiple on-chain DAOs where these fields set to 0. As a result, the protocol will fail to enforce a strictly positive liquidity invariant when transitioning from `Spot` to `Futarchy`. Because reserves are split utilizing integer division, a manipulated reserve of one truncates to zero.

Proof of Concept

The below steps illustrate a possible scenario that results in freezing the DAO:

1. An attacker utilizes a flash loan to swap a large amount against the spot pool, pushing `spot.base_reserves` down to one.
2. The attacker then immediately calls `launch_proposal`. During reserve splitting, the value rounds down to zero due to direct integer division, and the zero-threshold check still passes since no minimum reserve floor is enforced prior to `launch_proposal`.
3. Consequently, both pass and fail pools are initialized with `base_reserves = 0`, rendering swaps impossible. Additionally, `finalize_proposal` fails when the oracle aggregator becomes zero, effectively freezing the DAO as swaps are disabled and no new proposals can be initiated.
4. Users will not be able to withdraw their liquidity since the DAO AMM is stuck in futarchy state.

Remediation

Ensure that `min_base_futarchic_liquidity` and `min_quote_futarchic_liquidity` are non-zero.

Patch

Resolved in PR#419.

© 2026 Otter Audits LLC. All Rights Reserved.

PDA Authority Mismatch MEDIUM OS-MCV-ADV-02

Description

In `provide_liquidity`, the `amm_position` PDA is derived utilizing `params.position_authority` as a seed, but the account was initialized with `liquidity_provider.key` as the stored `position_authority`.

This creates a mismatch between the PDA derivation and the authority recorded in account state. Because `withdraw_liquidity` enforces both seed derivation and a `has_one = position_authority` constraint, the legitimate authority may fail withdrawal if the stored authority differs from the seed-derived key.

Thus, any one may provide liquidity to another user's `AmmPosition`, creating a temporary denial of service in which withdrawals revert due to an authority mismatch. Although the position authority may call deposit before withdraw to avoid this issue, so the issue might becomes more severe when integrated with other programs where the position authority itself is a PDA.

Remediation

Set `params.position_authority` to `position_authority` instead of `liquidity_provider.key`.

Patch

Identified and resolved by client in PR#418.

© 2026 Otter Audits LLC. All Rights Reserved.

Pool Initialization with Zero Base Reserves MEDIUM OS-MCV-ADV-03

Description

In `provide_liquidity`, when `total_liquidity == 0`, the bootstrap path does not enforce `max_base_amount > 0`, allowing the first depositor to initialize the pool with `base_reserves = 0`. This violates AMM invariants requiring both reserves to be non-zero. Thus, if zero base amount is provided, the DAO's `base_reserve` will be zero, resulting in swaps no longer working due to division by zero.

Remediation

Enforce `max_base_amount > 0` when `total_liquidity` is zero.

Patch

Resolved in PR#419.

© 2026 Otter Audits LLC. All Rights Reserved.

Lack of Dust Collection Mechanism MEDIUM OS-MCV-ADV-04

Description

In `redeem_tokens`, payouts are computed utilizing integer division, which rounds down and leaves a remainder of underlying tokens in the vault on each redemption. This residual dust may be up to `payout_denominator` units per redemption and is never paid out to users. Because redemptions are processed individually, these remainders accumulate over time. With multiple redemptions and low-decimal underlying tokens, the dust may accumulate into a significant non-negligible amount.

Remediation

Add a permissioned instruction to collect the accumulated dust from the vault.

Patch

This issue was acknowledged.

© 2026 Otter Audits LLC. All Rights Reserved.

Vault Lock Due to Excessive Outcome Count LOW OS-MCV-ADV-05

Description

There is a mismatch in account requirements between initialization and interaction instructions. `initialize_conditional_vault` succeeds with fewer accounts per outcome, while `split_tokens`, `merge_tokens`, and `redeem_tokens` require both the conditional mint and the user token account for each outcome. Thus, during initialization, `initialize_conditional_vault` only requires one account per outcome (the conditional token mint), allowing it to succeed even when the number of outcomes is relatively high.

However, split, merge, and redeem operations must simultaneously pass both the conditional token mint and the user's associated token account for each outcome. Consequently, if the number of outcomes falls within a specific range (such as 13-23), Solana's accounts-per-transaction limit will allow `initialize_conditional_vault` to succeed, but split/merge/redeem instructions will fail, resulting in unnecessary locking of rent lamports.

Remediation

Enforce a maximum limit on the number of outcomes in `initialize_question`.

Patch

Identified and resolved by client in PR#397.

© 2026 Otter Audits LLC. All Rights Reserved.

05 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-MCV-SUG-00	Using `Account<'info, Dao>` instead of `Box<Account<'info, Dao>>` risks stack overflows as the `Dao` structure grows.
OS-MCV-SUG-01	The code may be optimized by eliminating redundant ATA validations in high-frequency instructions, relaxing unnecessary ATA constraints, closing unutilized conditional vault accounts after proposal finalization, and short-circuiting arbitrage execution when no profitable opportunity exists.
OS-MCV-SUG-02	Suggestions for adding overflow checks, enforcing a non-zero `twap_max_observation_change_per_update` to preserve oracle liveness, and preventing split/merge execution after proposal resolution to maintain proper state transitions and governance correctness.
OS-MCV-SUG-03	The code may be refactored by moving fee parameters into DAO-configurable state, centralizing constants, and making fee collection permissionless since it transfers to fixed destinations.
OS-MCV-SUG-04	Suggestions regarding inconsistencies in the codebase and ensuring adherence to coding best practices.

© 2026 Otter Audits LLC. All Rights Reserved.

Inconsistent DAO Account Boxing OS-MCV-SUG-00

Description

`withdraw_liquidity`, `collect_fees`, `collect_meteora_damm_fees`, and `update_dao` instructions utilize `Account<'info, Dao>`, that deserializes the entire `Dao` structure onto the stack, which may exceed Solana's stack limit as the structure grows. Since `Dao` embeds large state, this creates risk of stack overflow in those instructions.

Remediation

Utilize `Box<Account<'info, Dao>>`, which moves the data to the heap and ensures consistency with other instructions.

Patch

Resolved in PR#419.

© 2026 Otter Audits LLC. All Rights Reserved.

Code Optimization OS-MCV-SUG-01

Description

1. Currently, high-frequency instructions such as `provide_liquidity`, `withdraw_liquidity`, spot/conditional swap, `finalize_proposal`, and `collect_fees` re-derive and re-validate the DAO's base and quote ATAs (`amm_base` and `quote_vault`) via Anchor's associated token constraints, adding unnecessary compute overhead on every call. Specifically, frequently invoked instructions such as swaps incur unnecessary compute costs when ATA checks are performed on every call. Instead, the vault addresses should be stored in the `Dao` account and used for direct pubkey validation.
2. Requiring `staker_base_account` to be the canonical ATA in `unstake_from_proposal` and `stake_to_proposal` is unnecessarily restrictive. The program only needs to verify that the token account's `mint == dao.base_mint` and `owner == staker.key`, since the SPL Token program enforces signature-based authority during transfers.
3. After `finalize_proposal` redeems all conditional tokens, the pass/fail AMM vault ATAs should hold zero balances and are no longer required. Close `amm_pass_base_vault`, `amm_pass_quote_vault`, `amm_fail_base_vault`, and `amm_fail_quote_vaul` to reclaim their rent-exempt lamports.
4. In both `arbitrage_after_spot_swap` and `arbitrage_after_conditional_swap`, within `futarchy_amm`, if no profitable input amount is found, `best_input_amount` remains zero. The current code still executes swaps with zero input, which is unnecessary. Add an early return when `best_input_amount` is zero.

Remediation

Include the above stated optimizations.

Patch

1. Issue #4 was resolved in PR#419.

© 2026 Otter Audits LLC. All Rights Reserved.

Missing Validation Logic

OS-MCV-SUG-02

Description

1. The `payout_denominator` in `resolve_question::handle` is computed utilizing an unchecked sum over user-provided `u32` values. If the total exceeds the maximum value of the denominator's integer type (`u32`), it will result in an overflow. It will be safer to utilize `checked_add` which performs addition while checking for overflow, instead of silently wrapping around.

>_ conditional_vault/src/instructions/resolve_question.rs RUST

```rust
impl ResolveQuestion<'_> {
    pub fn handle(ctx: Context<Self>, args: ResolveQuestionArgs) -> Result<()> {
        let question = &mut ctx.accounts.question;
        [...]
        question.payout_denominator = args.payout_numerators.iter().sum();
        question.payout_numerators = args.payout_numerators.clone();
        [...]
    }
}
```

2. In `finalize_proposal`, `pass_threshold_bps` and `team_sponsored_pass_threshold_bps` are added to `MAX_BPS` before widening, resulting in a possibility of an overflow occurring. Add overflow checks to prevent `pass_threshold_bps` and `team_sponsored_pass_threshold_bps` from overflowing.
3. If `twap_max_observation_change_per_update` is set to zero, the TWAP oracle cannot adjust from its initial observation, effectively freezing market price updates. Ensure that `twap_max_observation_change_per_update` is non-zero to prevent the DAO from becoming stuck after a proposal is launched.
4. After a question is resolved, conditional tokens have fixed payouts, so split and merge no longer have a meaningful role. Thus, it will be appropriate to ensure that the question is not resolved when executing split or merge instructions, so they cannot be invoked after resolution. While this has no direct security impact, it is advisable to enforce if there is no specific reason to allow such calls.

Remediation

Add the missing validation checks.

© 2026 Otter Audits LLC. All Rights Reserved.

Patch

1. Issue #1 was resolved in PR#397.
2. Issue #2 was resolved in PR#419.
3. Issue #3 was resolved in PR#419.
4. Issue #3 was resolved in PR#419.

© 2026 Otter Audits LLC. All Rights Reserved.

Code Refactoring OS-MCV-SUG-03

Description

1. Refrain from hardcoding `PROTOCOL_TAKER_FEE_BPS` and `LP_TAKER_FEE_BP` in `futarchy_amm` . Instead, store them on `Dao` state and add functionality in `update_dao` to update these values for easier configuration.

   >_ jup-sdk/src/futarchy_amm.rs RUST

   ```rust
   // use crate::{FutarchyError, LP_TAKER_FEE_BPS, MAX_BPS, PROTOCOL_TAKER_FEE_BPS};
   pub const LP_TAKER_FEE_BPS: u16 = 25;
   pub const PROTOCOL_TAKER_FEE_BPS: u16 = 25;
   ```

2. The current implementation utilizes string literals for PDA seeds. Define seeds as constants to improve maintainability. Similarly, the values in `Dao::invariant` should be defined as constants in a separate file to simplify future modifications.
3. `collect_fees` only transfers protocol-accumulated fees to fixed destination accounts owned by MetaDAO's multisig, with no caller-controlled parameters that may be manipulated. Thus, the instruction may be made permissionless without nay risks.

Remediation

Incorporate the above refactors.

Patch

1. Issue #2 was resolved in PR#419.

© 2026 Otter Audits LLC. All Rights Reserved.

Code Maturity OS-MCV-SUG-04

Description

1. The `decimals` field in `ConditionalVault` structure appears unutilized and may be removed.
2. `futarchy_amm::update_twap` currently utilizes the variable name `slot_difference` to represent the elapsed time between updates. This is misleading and it should be renamed to `time_difference` to accurately reflect that it measures a time interval in seconds, not slots.

>_ *programs/futarchy/src/state/futarchy_amm.rs* RUST

```rust
pub fn update_twap(&mut self, current_timestamp: i64) -> Result<Option<u128>> {
    [...]
    let new_aggregator = if current_timestamp <= twap_start_timestamp {
        oracle.aggregator
    } else {
        // so that we don't act as if the first update ocurred over the whole
        // pre-start delay period
        let effective_last_updated_timestamp =
            oracle.last_updated_timestamp.max(twap_start_timestamp);

        let slot_difference: u128 = (current_timestamp - effective_last_updated_timestamp)
            .try_into()
            .unwrap();

        // if this saturates, the aggregator will wrap back to 0, so this value doesn't
        // really matter. we just can't panic.
        let weighted_observation = new_observation.saturating_mul(slot_difference);
        oracle.aggregator.wrapping_add(weighted_observation)
    };
    [...]
}
```

3. `stake_to_proposal` currently assumes the proposal's base token ATA already exists, resulting in the instruction to fail if its not been pre-initialized. Utilize `init_if_needed` to ensure the escrow vault is created automatically on first stake.
4. `Dao` and its embedded `FutarchyAmm` both store `base_mint` and `quote_mint`, creating redundancy. Remove these fields from either `Dao` or from `FutarchyAmm`.

Remediation

Implement the above-mentioned suggestions.

© 2026 Otter Audits LLC. All Rights Reserved.

Patch

1. Issue #2 was resolved in PR#419.

© 2026 Otter Audits LLC. All Rights Reserved.

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
- Improperly designed economic incentives leading to loss of funds.

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
- Exploitation involving high capital requirement with respect to payout.

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
- Forced exceptions in the normal user flow.

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
- Improved input validation.

© 2026 Otter Audits LLC. All Rights Reserved.

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.

© 2026 Otter Audits LLC. All Rights Reserved.